

DLCA Lab 4: Simulator

Week 1

August 26, 2025

1 Introduction

In this week's lab, you shall be learning how to use a state-of-the art microarchitectural simulator (Champsim) to see the effects of different microarchitectural policies on different workloads.

2 Necessary tools

We shall be using make and g++ in order to compile C++ code.

Before starting the lab, ensure that the tool is installed in your system. This can be verified by running the following:

```
$ make
make: *** No targets specified and no makefile found. Stop.
$ g++
g++: fatal error: no input files
compilation terminated.
```

You will also need to ensure that your system has the traces installed, this can be checked by running:

```
$ ls $CS231_TRACE_DIR
bwaves.xz gcc.xz mcf.xz sssp.gz
```

You must also ensure that Champsim has been installed and setup in the Desktop folder of your lab machine, there must be a folder on your desktop called Champsim. You can ensure it is correct by running the following from your desktop:

```
ls Champsim/
bin branch btb champsim_config.json CITATION.cff config.sh
inc LICENSE Makefile obj prefetcher replacement run_champsim.sh src storage tracer
```

3 Structure of submission/templates

```
your-roll-no/
|- task1/
|   |- random.cc
|- task2/
|   |- rrip.cc
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz`. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

4 Tasks

This assignment consists of two subtasks. These tasks are expected to be submitted in-lab. The first task is intended to get you comfortable with modifying code in Champsim while the second one is intended for you to make a meaningful modification to a replacement policy. Champsim can be a bit overwhelming to work with, which is why we will be holding your hand through the first task and getting you through it. Our goal as TAs is to ensure every student attending the lab is able to solve task 1.

Some background

How to build Champsim?

In this lab, we want to focus on LLC Cache replacement policies in Champsim. Therefore, we will be building champsim with specific LLC replacement policies. Suppose we want to build champsim with the random replacement policy at the LLC, to do this, you must ensure the following:

1. In the Champsim/replacement/ folder, there must be a folder called random/ with a file called random.cc
2. random.cc must contain the following functions:
 - (a) void CACHE::initialize_replacement
 - (b) uint32_t CACHE::find_victim
 - (c) void CACHE::update_replacement_state
 - (d) void CACHE::replacement_final_stats
3. In the Champsim/ folder, the file champsim_config.json must have "executable_name": "bin/lru" and

```
"LLC": {  
    ....  
    "replacement": "random"  
},
```

4. Run `./config.sh champsim_config.json && make`

This builds the champsim binary with the required random replacement policy.

How to run Champsim?

We have given you a tool to run Champsim, the tool takes in two arguments:

1. TRACE_FILE: This is the workload which is run on the simulator, you have been given four workloads which are found in the `$CS231_TRACE_DIR` folder
2. BINARY: This is the specific champsim binary which contains the simulator on which to run the workload

An example run of champsim looks like:

```
$ bash run_champsim.sh $CS231_TRACE_DIR/sssp.gz random

*** ChampSim Multicore Out-of-Order Simulator ***

...
```

The replacement policy functions

To implement your replacement policy you will need to understand what the replacement policy functions are:

1. `void CACHE::initialize_replacement`: This function is called before simulation begins. It is there so that you can set up your data structures that you may be using.
2. `uint32_t CACHE::find_victim`: This function is called at the time that an eviction needs to occur from the cache set. You need to return the way number of the cache line to be evicted.
3. `void CACHE::update_replacement_state`: This function is called whenever there is a cache hit to a block and whenever a block is filled into the cache.

Task 1: A random replacement policy

A random replacement policy is one in which the candidate cache line to be evicted is selected at random. There is no promotion or insertion policy since all lines have an equal probability of being evicted.

You can sample random numbers using the `rand()` function. We will be implementing this policy live with you, so please follow along with us :). We will be looking at the Cumulative IPC and LLC Misses per Kilo Instructions (MPKI) which is given by $MPKI = \frac{Misses * 1000}{Instructions}$. The expected output can be seen in table 1

Workload	Cumulative IPC	LLC Total Misses
bwaves	0.353013	950176
gcc	0.324657	888642
mcf	0.439848	681624
sssp	0.326941	1138407

Table 1: Simulation statistics of different workloads when using random replacement policy at the LLC

Task 2: Re-Reference Interval Prediction replacement policy

In this task you will be implementing the Re-Reference Interval Prediction (RRIP) replacement policy. The RRIP policy requires that every cache line has 3 bits associated with it, called as the `rripBits` which have a value called the `rripVal`. The replacement policy works as follows:

1. Insertion Policy: Insert with the `rripVal` set to 1
2. Promotion Policy: On a hit, set the `rripVal` set to 7
3. Eviction Policy: Evict randomly from among the cache ways having 0 as the `rripVal`. If there are no such lines, proceed as follows:
 - (a) Suppose the lowest `rripVal` among the cache lines in the set is x
 - (b) For each cache line, if it had an `rripVal` of y , change it's `rripVal` to $y - x$
 - (c) Evict randomly from among the cache ways having 0 as the `rripVal`

Workload	Cumulative IPC	LLC Total Misses
bwaves	0.353013	950160
gcc	0.324694	888448
mcf	0.497402	551317
sssp	0.364933	872441

Table 2: Simulation statistics of different workloads when using rrip replacement policy at the LLC

The expected output for the rrip policy can be seen in table 2.